PortProof

Architecture document · verified SQL migration to IBM Db2

PortProof translates PostgreSQL and MySQL statements to Db2 and then makes **Db2 itself** establish that the result is correct — first that it compiles, through a real server-side `PREPARE`, then that it returns the same rows, through differential execution against a live PostgreSQL and a live MySQL over six adversarial datasets. The application proposes. The engine decides. Nothing in this system reports a verdict that the database did not produce.

THE PROBLEM

Every SQL migration tool reports whether it *translated* a statement. None reports whether the answer is still correct. Those are different questions, and the gap between them is where migrations quietly go wrong.

Measured here, on a 38-statement corpus drawn from real migration patterns, using IBM's own PostgreSQL-to-Db2 translator (`db2-sqlglot-dialect`):

38
statements translated, with **zero** errors reported by the translator

14
rejected outright by Db2 the moment it was asked to compile them

7
compiled cleanly and returned **different rows** than the source engine

0
of those 21 problems were visible to anyone reading the tool's output

The second category is the dangerous one. A migration that errors is an inconvenience; a migration that succeeds and returns subtly different numbers is a report nobody questions. There is no stack trace, no alert and nothing to triage — a case-insensitive join or a changed `GROUP BY` simply produces a plausible wrong answer. PortProof exists to close that gap, and it closes it by refusing to be the thing that decides.

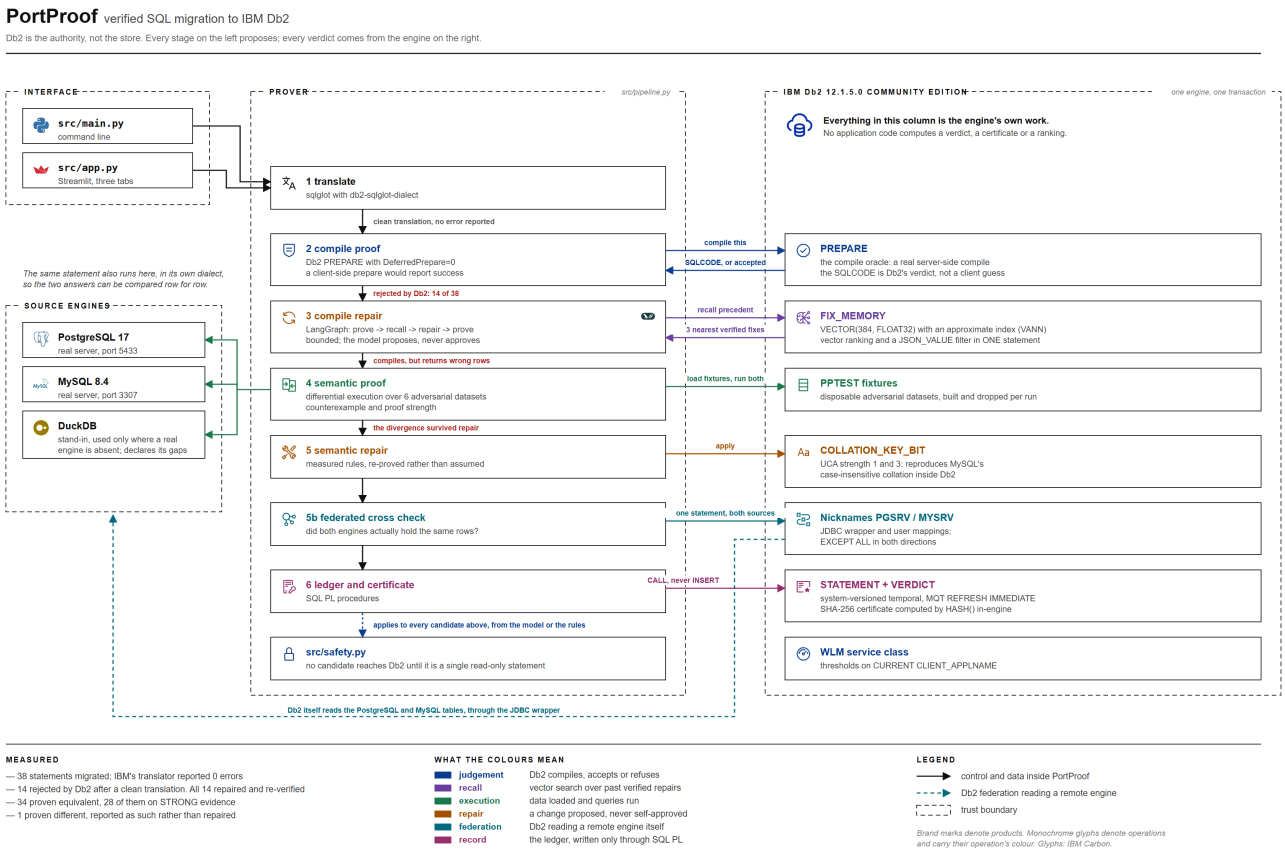
DESIGN PRINCIPLE

> **Db2 is the authority, not the store.** Most applications treat a database as somewhere to keep results. Here the engine is the one component permitted to reach a conclusion. Every stage of the pipeline produces a *proposal*; every verdict, every certificate and every ranking is computed inside Db2 and can be re-derived from evidence the system shows you.

AT A GLANCE

ASPECT	DETAIL
Engine under test	IBM Db2 Community Edition 12.1.5.0 (Docker), the authority for every verdict
Source engines	PostgreSQL 17 and MySQL 8.4, live, for differential execution and federation
Third engine	DuckDB, stand-in only, used where a real engine is not needed to make the point
Entry points	`src/main.py` (CLI, the full pipeline) and `src/app.py` (Streamlit, three tabs)
Identities	`PPOWN` owns the schema; `PPAPP` runs the pipeline and cannot write a row
Repair proposer	deterministic rules by default; optional LLM via `LLM_PROVIDER`, grounded by vector recall
Corpus	38 statements, 6 adversarial datasets, 151 tests

ARCHITECTURE



Each prover stage shares a baseline with the Db2 object it talks to, so every crossing arrow is horizontal: work happens in the middle column, judgement in the right one. Arrow labels are tinted by kind of operation; the labels down the prover spine by severity. The drawing reads correctly in greyscale — colour is a redundant second layer, never the only carrier of meaning.

THE STAGES, AND WHAT EACH IS NOT ALLOWED TO DO

1 · Translate

`sqlglot` with `db2-sqlglot-dialect`. Deliberately not improved upon: the project's claim is about what IBM's own translator leaves unverified, so replacing it would remove the thing being measured.

2 · Compile proof

Db2 `PREPARE` with **`DeferredPrepare=0`**. This is not cosmetic. With the Db2 CLI default, `PREPARE` is held on the client until execution, so `ibm_db.prepare()` returns success for SQL the server would reject — measured on CE 12.1.5. Switching deferred prepare off is what makes the SQLCODE the engine's own verdict rather than a client's guess. This stage is what catches the 14.

3 · Compile repair

A bounded LangGraph state machine: `prove → recall → repair → prove`. A language model proposes a fix, grounded in repairs Db2 has already verified and recalled by cosine distance from a vector column inside the database. **The model cannot approve its own work.** Every candidate is checked by `src/safety.py` to be a single read-only statement before it reaches a connection, and is then compiled by Db2. The loop can only exit through the engine, and it is bounded, so a model that cannot fix something fails loudly rather than looping.

The proposer is swappable — a deterministic rule engine by default (no API key, so every number here is reproducible), or a real LLM via `LLM_PROVIDER=gemini` or `http`. **The verdicts do not move when it is swapped**, because the proposer never casts one.

4 · Semantic proof

Compiling proves nothing about the answer. The original statement runs on a real PostgreSQL or MySQL; the translation runs on Db2; both run over six datasets built specifically to break translations (mixed case, ties and duplicates, NULL-heavy, unicode, boundary numerics, empty). Differences are returned as a **counterexample**: the exact dataset and both result sets, so a reader can check the verdict instead of believing it.

Each verdict carries a **proof strength** (`NONE / WEAK / MODERATE / STRONG`), because two empty result sets agreeing is not evidence of equivalence. An earlier version of this project certified a statement as equivalent on exactly that basis; the strength score and the vacuity check exist because of it.

5 · Semantic repair

Measured rewrite rules, re-proved rather than assumed. The headline case is MySQL's case-insensitive default collation, reproduced inside Db2 with `COLLATION_KEY_BIT` at UCA strength 1 and 3.

5b · Federated cross-check

One question a comparison written in Python can never answer: when two engines disagree, is that the query meaning something different, or did the harness load slightly different data into each? A timestamp that lost its fractional seconds looks exactly like a mistranslated `GROUP BY`. Federation nicknames over the PostgreSQL and MySQL tables let a **single Db2 statement** read both copies and compute the symmetric difference with `EXCEPT ALL` in both directions. A table it could not read is reported as `UNAVAILABLE`, never folded into a pass.

6 · Ledger and certificate

The application identity holds **no INSERT on any table**; it can only `CALL` SQL PL procedures. Each verdict's certificate is a SHA-256 computed by `HASH(..., 2)` *inside the engine*, over the evidence rather than the conclusion, so the application never chooses it. The ledger is system-versioned, so "what did this say sixty seconds ago" is answered by Db2 from its own history.

TRUST BOUNDARIES

COMPONENT	TRUSTED TO	EXPLICITLY NOT TRUSTED TO
Translator	produce a candidate	say whether it is valid or correct
Language model	propose a repair	approve one, or run anything directly
`src/safety.py`	reject non-read-only candidates	decide a statement is correct
Application (`PPAPP`)	read, and call procedures	insert, update, or write a certificate
Owner (`PPOWN`)	DDL and fixture setup	run the proof pipeline; it is not DBADM
Db2	**every verdict, certificate and ranking**	—

WHAT THE ENGINE ACTUALLY DOES

Twenty-one Db2 capabilities, each load-bearing: remove any one and a claim the project makes stops being provable.

AREA	CAPABILITY
Compile oracle	`PREPARE` server-side with `DeferredPrepare=0`; SQLSTATE and SQLCODE taken as the verdict
Vector / AI	`VECTOR(384, FLOAT32)`; `CREATE VECTOR INDEX ... WITH DISTANCE COSINE` (INDEXTYPE `VANN`); `VECTOR_DISTANCE`; `FETCH APPROX FIRST`
Hybrid retrieval	vector ranking and a `JSON_VALUE` metadata predicate evaluated in **one statement**, planned by one optimiser
JSON	`JSON_VALUE(... RETURNING ...)` typed extraction

Integrity	`HASH(..., 2)` SHA-256 computed in-engine over the evidence
Temporal	system-versioned tables; `FOR SYSTEM_TIME AS OF`
Aggregation	MQT with `REFRESH IMMEDIATE MAINTAINED BY SYSTEM`
Procedural	SQL PL procedures; `MERGE` upsert; cursor loop with `CONTINUE HANDLER FOR NOT FOUND`
Collation	`COLLATION_KEY_BIT` at `UCA500R1_S1` and `UCA500R1_S3`
Federation	JDBC wrapper; `POSTGRESQL` and `MYSQL` servers; user mappings to read-only remote accounts; nicknames; `EXCEPT ALL` both directions
Workload	WLM service class and workload matched on `CURRENT CLIENT_APPLNAME`; `SQLROWSRETURNED` and `ESTIMATEDSQLCOST` thresholds
Physical	32K bufferpool and tablespace (a 384-dimension vector index key does not fit a 4K page); `ADMIN_GET_TAB_COMPRESS_INFO`; `MON_GET_PKG_CACHE_STMT`; `ORGANIZE BY COLUMN` check
Security	two-identity least-privilege split enforced by `GRANT`; expression-based index; CHECK, UNIQUE and FK `ON DELETE CASCADE`

WHY DB2, AND NOT DB2 PLUS A VECTOR DATABASE

The fix memory is the clearest case. Recalling a past repair requires a semantic ranking *and* a relational predicate at once. In PortProof that is one statement against one engine inside one transaction:

```
SELECT JSON_VALUE(METADATA, '$.fixed_sql' RETURNING VARCHAR(4000)),
       VECTOR_DISTANCE(EMBEDDING, VECTOR(?, 384, FLOAT32), COSINE) AS DIST
  FROM FIX_MEMORY
 WHERE JSON_VALUE(METADATA, '$.dialect' RETURNING VARCHAR(16)) = ?
 ORDER BY DIST
 FETCH APPROX FIRST 3 ROWS ONLY
```

Bolt a separate vector store onto Db2 and this becomes two systems, two consistency problems and two sets of permissions, with the filter applied either before or after the ranking rather than planned with it. Here the embeddings live in the same table as the metadata, under the same `GRANT`s, in the same unit of work.

MEASURED RESULTS

14 / 14
silent compile failures repaired and **re-verified by Db2**

34
statements proven equivalent, 28 on STRONG evidence

1
proven different, reported as such rather than repaired

63.4%
fewer pages, by Db2's own administrative function

Also: 2 statements whose order is not determined by the SQL (reported, not guessed), 0 vacuous passes, 1 inconclusive. Federation confirmed both engines held identical rows across all four fixture tables, so the divergences found were differences in meaning and not in data. **151 tests pass.** The default repair engine needs no API key, so these figures are reproducible by anyone who clones the repository and runs `python -m src.main`.

WHAT IT DOES NOT DO

- **It proves statements, not schemas.** No DDL migration, no constraint or trigger translation.
- **Equivalence is empirical, not formal.** Six adversarial datasets are evidence, not a proof over all inputs. That is precisely why every verdict carries a strength score, and why two empty result sets agreeing is reported as weak rather than as a pass.
- **One divergence remains unrepaired.** `postgres/column_division`: a genuine difference between PostgreSQL and Db2 decimal division. The report names it and explains it rather than papering over it.

- **The corpus is 38 statements.** Enough to demonstrate the method and to find 21 real defects; not a production estate.
